The next function, *Figure 5.18*, drops the records that are more than `windowMinutes` old from `TransactionCountState` by calculating the state expiration timestamp (the latest timestamp minus the transaction count minutes). Then, it loops through the list of current transactions and keeps any that occur before the expiration timestamp. This is shown in the following screenshot:

```
def dropExpiredRecords(newLatestTimestamp: java.time.Instant,
    currentTransactions: List[InputRow]): TransactionCountState = {
    val newTransactionList = ListBuffer[InputRow]()
    val expirationTimestamp = Instant.ofEpochMilli(newLatestTimestamp.
    toEpochMilli() - TimeUnit.MINUTES.toMillis(windowMinutes))

    if (currentTransactions.size > 0) {
      currentTransactions.foreach { value =>
        if (value.TransactionTimestamp.toEpochMilli() >=
        expirationTimestamp.toEpochMilli())
          newTransactionList.append(value)
      }
    }

    new TransactionCountState(newLatestTimestamp,
                              newTransactionList.toList)
  }
```

Figure 5.18 – Defining a function to drop stale records

The `updateState` function uses our helper functions to – you guessed it – update the customer state. This is the function called `flatMapGroupsWithState`. The `updateState` function receives the customer ID, values, and the current state. `CustomerID` is the key we are grouping on. `values` is an iterator of `InputRow`. In *Figure 5.6*, we see that `InputRow` is a transaction record consisting of a `CustomerID` reference and the time the transaction occurred. The `updateState` function behaves in two ways. Suppose one or more `InputRow` records for a given `CustomerID` are received. In that case, it will add those records to the state, drop any records that are older than `windowMinutes` from the state, and calculate the transaction count. See *Figure 5.13* for the notebook code.